

Numerical Semigroups, Formalized

A guide for the team — what we're building, and how to set up your workspace

DATE July 2026

§1 The project in a nutshell

We are going to teach a computer a piece of mathematics it does not yet know, prove a real theorem inside it so that *every single step* is checked by the machine, and then use that same machinery to fact-check the numbers printed in published research papers. Three concrete goals: build a **library**, prove a **theorem**, and run an **audit** of the literature. Everything we make is public, reusable, and honest about what has and hasn't been verified.

§2 The mathematics you'll be working with

A puzzle to start. You have coins worth 3, 5, and 7. Which exact totals can you pay? You can make 0, 3, 5, 6, 7, 8, ... — everything from 5 upward — but never 1, 2, or 4. The set of makeable totals is the **numerical semigroup** $\langle 3, 5, 7 \rangle$. The unmakeable numbers $\{1, 2, 4\}$ are its **gaps**; there are 3 of them (the **genus**), and the largest, 4, is the **Frobenius number** — the biggest total you can never form.

Formally. A numerical semigroup is a set of non-negative integers containing 0, closed under addition, that misses only finitely many numbers. It is always $\langle n_1, \dots, n_k \rangle$ — all non-negative combinations of some positive generators whose gcd is 1.

Factorization is non-unique. Inside a numerical semigroup, one element can be built in several ways with *different lengths*. In $\langle 2, 3 \rangle$, the number 6 is $2+2+2$ (length 3) or $3+3$ (length 2). The set of all lengths of an element is its **length set**. Measuring how badly length fails to be unique is the subject of **factorization theory**.

Elasticity. The elasticity $\rho(n) = \max L(n) / \min L(n)$, and $\rho(S)$ is the largest value over all elements. Our **flagship theorem** gives it a beautiful closed form: for $S = \langle n_1 < \dots < n_k \rangle$,

$$\rho(S) = \frac{n_k}{n_1}.$$

Why it's true (intuition). A factorization of n into L pieces uses L generators, each between n_1 and n_k , so $L \cdot n_1 \leq n \leq L \cdot n_k$ — every length sits in $[n/n_k, n/n_1]$. The extreme ratio n_k/n_1 is hit exactly at $n = n_1 \cdot n_k$.

§3 What “formal proof” means, and our tools

A **proof assistant** is software where every logical step is checked against the axioms of mathematics — if the file compiles, the theorem is correct, with no “it is easy to see.” We use **Lean 4** and its huge library **Mathlib**. Lean also *computes*: for a finite claim like “the Frobenius number of $\langle 3, 5, 7 \rangle$ is 4,” the tactics `decide` and `native_decide` evaluate it and certify it — which is exactly how we audit a paper’s tables.

Both `decide` and `native_decide` prove a claim by actually *computing* a yes/no answer instead of reasoning it out by hand, so choosing between them is a trade-off between speed and how much you have to trust.

- **decide** — Lean’s tiny, trusted proof-checking kernel runs the computation itself and confirms the result, so a successful proof adds no new assumptions beyond that kernel you already rely on (you can see this with `#print axioms`, which reports none). It’s airtight but comparatively slow, which makes it the natural first choice for small checks like verifying that the Frobenius number of $\langle 3, 5, 7 \rangle$ is 4.
- **native_decide** — compiles the same yes/no check to fast machine code, runs it, and then simply asserts the answer it got as a brand-new *axiom* baked into your proof (its name contains `native_decide`; older Lean versions instead recorded this trust under one called `ofReduceBool`). That widens your circle of trust to include Lean’s compiler and runtime, not just its kernel.
- **How we use them** — we prefer `decide` whenever it finishes in reasonable time, and fall back to `native_decide` only for computations too big for the kernel to grind through. Running `#print axioms` then shows exactly what a proof relied on (nothing extra for `decide`, versus that one `native_decide` axiom), and we re-check every headline number in GAP, so any single bug would have to fool two independent systems to slip past us.
- **GAP + numericalsgps** — a computer-algebra system and its numerical-semigroups package; our independent second opinion for every value we compute.
- **Aristotle** — an AI that drafts Lean proofs. Its output is still re-checked by the same tiny kernel, so it can never introduce an unsound step; it only saves time.

§4 Our three goals

Goal	What it means
1 • Library	Teach Lean about numerical semigroups and their invariants — membership, gaps, Frobenius number, genus, factorizations, length sets, elasticity. (<i>The computational core already builds and is self-tested.</i> DONE)
2 • Theorem	Formally prove the flagship elasticity formula $\rho(S) = n_k/n_1$, with no gaps.
3 • Audit	Re-derive the concrete numbers in ~ 15 – 20 published papers with <code>decide</code> / <code>native_decide</code> , cross-check every value in GAP, and report any discrepancy. Catching even one published-table error would be a highlight.

§5 How we'll divide the work

Two threads run in parallel. Rather than fix dates, we fix **ownership**:

- **The theory thread** — building the library and proving the flagship theorem (Lean).
- **The audit thread** — encoding papers' numeric claims, cross-checking GAP, and logging results.

Suggested plan: each of you leads one thread, then **swap** partway through so you both learn both sides. The two threads are independent enough to work on at the same time — the audit needs only the finished computational core, not the finished proof. Your mentor reviews regularly, and GAP is the independent oracle for every number.

§6 Setting up your repositories

Create two repositories. The right dividing line is “*code that must compile*” vs. “*everything else*” — not simply Lean vs. non-Lean.

Repo 1 – the Lean library (must build; CI-green). The lake project at its root, so builds, CI, and a possible Mathlib contribution stay clean:

- lakefile.toml, lean-toolchain, NumSemigroups/ (Defs, Factorization, Elasticity).
- The audit's Lean checks (Verify/PaperClaims.lean) — they import the library, so they live here.
- A CI action that runs lake build.

Repo 2 – the project / research repo (nothing needs to compile).

- The write-up (the Yau paper — $\text{\LaTeX}$ or Overleaf).
- The GAP cross-check scripts.
- The audit results table: paper $\rightarrow$ claim $\rightarrow$ verdict (pass or discrepancy).
- Literature notes, the reading list, slides, meeting notes, admin.

6.1 Why two – and not one or three

- **Why not one repo?** A single “monorepo” works and is marginally simpler to clone, but it entangles the buildable library with prose and tools, which muddies CI and any Mathlib contribution.
- **Why not three or more?** For a two-person team, more repos means scattered issues, more context-switching, and a harder time seeing the whole project. Two is the sweet spot.
- **One thing to watch.** The audit spans both repos — Lean checks in Repo 1; GAP scripts, the results table, and the write-up in Repo 2. Put a link to each repo in the other's README so nothing gets lost.

6.2 Creating them

Suggested names: `ns-lean` and `ns-project`. Keep them private until you're ready to submit.

NEVER COMMIT SECRETS The Aristotle API key (and any other key) must stay out of git. Put it in a `.env` file and add a `.gitignore` that ignores `.env` (and, in the Lean repo, `.lake/` and build artifacts). If a key is ever committed, treat it as compromised and rotate it.

§7 Getting started

A short checklist to get from zero to your first machine-checked fact — do these in whatever order works, there's no clock on it:

1. Install VS Code + the Lean 4 extension (it installs the `elan` toolchain manager for you). Guide: https://leanprover-community.github.io/get_started.html
2. Create the two repositories (Section 6).
3. Warm up with the Natural Number Game (browser, no install): <https://adam.math.hhu.de/#/g/leanprover-community/NNG4>
4. In the Lean repo, run `lake exe cache get` (downloads prebuilt Mathlib — do this FIRST), then `lake build`.
5. Make one line compile in a scratch file:

LEAN

```
example : frobeniusUpTo [3,5,7] 20 = 4 := by native_decide
```

6. Cross-check it in GAP: `FrobeniusNumber(NumericalSemigroup(3,5,7))` returns 4. Two independent tools, same answer — that is our whole method, in miniature.

§8 Resources

LEARN LEAN Natural Number Game · Mathematics in Lean · all learning resources

GET HELP / SEARCH Lean Zulip chat · Mathlib docs · LeanSearch

TOOLS GAP `numericalsgps` · Aristotle

BOOKS Rosales & García-Sánchez, *Numerical Semigroups* (Springer 2009) · Geroldinger & Halter-Koch, *Non-Unique Factorizations* (2006).

8.1 The audit-target papers

Grouped by invariant family — the closed-form families are the best starting targets. The annotated version (with what to verify) is in the project repo's candidate-papers document.

Best starting targets – closed-form families.

- Fibonacci numerical semigroups — [arXiv:math/0606717](https://arxiv.org/abs/math/0606717)

- Generalized Thabit numerical semigroups — [arXiv:1510.04801](#)
- Generalized repunit numerical semigroups — [arXiv:2306.10738](#)
- Repunit numerical semigroups (Rosales–Branco–Torrão) — [Ramanujan J.](#)
- Semigroups from a linear recurrence — [arXiv:2111.04899](#)
- Frobenius number of geometric sequences — [op.pdf](#)
- Triangular & tetrahedral number sequences — [arXiv:1706.04378](#)
- Squares/cubes/quartics of three consecutive integers — [arXiv:1608.08693](#)
- Frobenius number of squares of a semigroup — [arXiv:2507.01898](#)

Small generator counts.

- Frobenius problem for four numerical semigroups — [arXiv:1706.09246](#)
- Arf numerical semigroups with given Frobenius number — [arXiv:2303.12470](#)
- Symmetric semigroups generated by three elements — [arXiv:0903.3756](#)

Elasticity.

- Set of elasticities in numerical monoids — [arXiv:1409.3425](#)
- Elasticity in Apéry sets — [arXiv:1908.06448](#)

Delta sets.

- Delta sets, embedding dimension three — [arXiv:1504.02116](#)
- Delta sets, symmetric embedding dimension three — [arXiv:1701.00988](#)
- Realization problem for delta sets — [arXiv:1503.08496](#)
- ∞ - and 0-delta sets — [arXiv:2503.12241](#)

Catenary / tame degree / ω -invariant.

- Catenary & tame degree, generalized arithmetic sequences — [arXiv:1001.1646](#)
- Dynamic algorithms for factorization invariants — [arXiv:1507.07435](#)
- Arithmetical invariants (semigroup-theoretical) — [arXiv:1006.4222](#)